Smart NPC — Implementation Plan

Project Overview

AI-powered conversational avatar for RP1's Open Metaverse Browser. Accepts dynamic personas and role-play scenarios (e.g., "salary negotiation with boss Brian"). Built incrementally in phased approach.

Phase 1: Speech-to-Text (STT) — COMPLETE

Goal: Transcribe speech from nearby avatars in real-time.

Stack: Deepgram v2 Flux API (`wss://api.deepgram.com/v2/listen`)

Architecture

```
MVRP Decode Interceptor → AudioFrameBuffer (ring buffer)
    → STTDrainLoop (10Hz timer, AGC normalization)
    → PCMConverter (mono, downsample 24k→16k, Int16)
    → STTService (WebSocket to Deepgram)
    → TranscriptEvent → UI
```

Key Files

File	Purpose
`src/ai/STTService.ts`	Deepgram v2 WebSocket client, TurnInfo event parsing
`src/ai/STTDrainLoop.ts`	Timer-based drain: buffer → PCMConverter → STTService
`src/audio/PCMConverter.ts`	stereoToMono, downsample, float32ToInt16, AGC
`src/audio/AudioFrameBuffer.ts`	Lock-free ring buffer for PCM samples
`src/audio/AudioFrameCapture.ts`	Decode interception bridge
`src/audio/ProximityAudioManager.ts`	MVRP decode interceptor, mono/stereo auto-detection
`src/client/InWorldSession.ts`	STT lifecycle (startSTT/stopSTT)

Key Discoveries

- MVRP decoded audio is **mono** at **24 kHz** (not stereo 48 kHz)
- Decode-stage output is very quiet (~0.05 peak) — AGC required
- Deepgram v2 only accepts `CloseStream` and `Configure` as text messages (no KeepAlive)
- `ScriptProcessorNode` path captures silence; only decode interception works

Tests: 56 passing

Phase 2: Text-to-Speech (TTS) — IN PROGRESS

Goal: Convert text to speech and play it through the avatar so nearby users hear it.

Stack: Deepgram TTS API (`wss://api.deepgram.com/v1/speak`)

Architecture

```
Text input → TTSService (WebSocket to Deepgram)
    → Binary PCM chunks (linear16 @ 48kHz)
    → int16ToFloat32 → AudioBuffer
    → Gapless scheduling via AudioContext
    → AVStreamAudioPlayer (local playback with spatial audio)
```

Deepgram TTS API Details

Parameter	Value	Reason
Endpoint	`wss://api.deepgram.com/v1/speak`	WebSocket for streaming low-latency
Encoding	`linear16`	Raw PCM, zero decode overhead
Sample Rate	`48000` Hz	Matches browser AudioContext default
Container	`none`	Raw bytes, no WAV header clicks
Auth	`?token=KEY` query param	Browser WebSocket can't set custom headers
Default Voice	`aura-2-thalia-en`	Clear, energetic female American

WebSocket Protocol

```
Client → { type: "Speak", text: "Hello world" }   // queue text (max 2000 chars)
Client → { type: "Flush" }                         // trigger audio generation
Server → <binary PCM chunk>                        // raw linear16 audio
Server → <binary PCM chunk>                        // more chunks...
Server → { type: "Flushed" }                       // all audio for this flush sent
Client → { type: "Close" }                         // graceful shutdown
```

Constraints

- Max 2000 characters per Speak message
- Max 20 Flush messages per 60 seconds
- One voice per WebSocket session
- 60-minute connection timeout

Available Voices (Aura-2)

Model	Gender	Accent	Character
`aura-2-thalia-en`	F	American	Clear, Energetic
`aura-2-apollo-en`	M	American	Confident, Casual
`aura-2-arcas-en`	M	American	Natural, Smooth
`aura-2-draco-en`	M	British	Warm, Trustworthy

`aura-2-zeus-en`	M	American	Deep, Trustworthy
`aura-2-pandora-en`	F	British	Smooth, Calm
`aura-2-aurora-en`	F	American	Cheerful, Expressive

Implementation Steps

1. **TTSService.ts** — Deepgram WebSocket client
   - Connect with token auth via query param
   - Send Speak/Flush messages
   - Receive binary PCM chunks
   - Gapless audio scheduling via AudioContext
   - Handle Flushed/Error/Warning control messages
2. **Config** — Add TTS settings to `config.ts` and `.env`
   - `DEEPGRAM_TTS_MODEL_DEFAULT` (voice selection)
   - `DEEPGRAM_TTS_SAMPLE_RATE_DEFAULT` (48000)
3. **InWorldSession** — TTS lifecycle
   - `startTTS()` / `stopTTS()` / `speak(text)`
   - Wire TTSService to AVStreamAudioPlayer for spatial playback
4. **UI** — Text input + speak button in index.html
   - Text input field
   - Speak button
   - Voice selector dropdown
   - TTS connection status badge
5. **Tests** — Unit tests for TTSService

Playback Strategy: Gapless Scheduling

```
// Queue chunks back-to-back using AudioContext timing
const startTime = Math.max(audioCtx.currentTime, this.nextPlayTime);
source.start(startTime);
this.nextPlayTime = startTime + audioBuffer.duration;
```

Existing Utilities (Already Built)

- `PCMConverter.int16ToFloat32()` — convert Deepgram PCM16 to Web Audio Float32
- `PCMConverter.resampleTo48k()` — upsample if Deepgram returns lower rate
- `AVStreamAudioPlayer.playBuffer()` — spatial audio playback with GainNode/PannerNode

Phase 3: Round-Trip Voice Loop

Goal: STT → echo → TTS (avatar repeats what it hears).

Architecture

```
Nearby avatar speaks → STT transcribes
    → Final transcript text
    → TTSService.speak(text)
    → Avatar speaks the same words back
```

Implementation

- Wire `InWorldSession.onTranscript` final events to `TTSService.speak()`
- Add toggle in UI: "Echo Mode"
- Add debounce to prevent feedback loops

Phase 4: LLM Integration

Goal: Replace echo with AI-generated responses.

Architecture

```
STT transcript → LLM (streaming) → TTS
```

Planned Stack

- LLM: OpenAI GPT-4o or similar (streaming completions)
- New `LLMService.ts` — manages conversation history, streams tokens
- Token-by-token streaming into TTSService for lowest latency

Key Design

- Stream LLM tokens directly to `TTSService.speak()` as they arrive
- Sentence-boundary detection for natural `flush()` timing
- Conversation history management (sliding window)

Phase 5: Persona Engine

Goal: Dynamic character profiles that shape LLM behavior.

Planned Features

- Persona definition schema (name, backstory, personality, voice, rules)
- System prompt builder from persona definition
- Voice mapping (persona → Deepgram voice model)
- Multiple concurrent personas

Phase 6: Guided Persona Gathering

Goal: Conversational setup flow where the user describes a scenario and the system builds a persona.

Planned Features

- Multi-turn conversation to gather persona details
- Auto-generate persona definition from user description
- "Salary negotiation with boss Brian" → full persona with backstory, personality, rules

- Save/load persona definitions

Audio Pipeline Reference

Inbound (MVRP → STT)

```
MVRP Decode (24kHz mono Float32, peak ~0.05)
    → AudioFrameBuffer ring buffer
    → STTDrainLoop (100ms interval)
    → AGC (normalize to RMS 0.15, max gain 50x)
    → downsample 24k → 16k
    → float32ToInt16
    → Deepgram STT WebSocket
```

Outbound (TTS → Speakers)

```
Deepgram TTS WebSocket
    → Binary PCM chunks (linear16 @ 48kHz)
    → int16ToFloat32
    → AudioBuffer (Web Audio API)
    → Gapless scheduling
    → AVStreamAudioPlayer (GainNode → PannerNode → destination)
```

Future: Broadcast to Other Users

```
TTS PCM output
    → Encode as MVRP codec 0 (PCM16) or codec 1 (delta)
    → UPDATE payload { wSamples, wCodec, abData }
    → pRPersona.Send('UPDATE', payload)
    → Server → other users' proximity audio
```

Environment Configuration

```
# Deepgram (shared key for STT + TTS)
DEEPGRAM_API_KEY=<key>
DEEPGRAM_API_ENDPOINT=wss://api.deepgram.com/v2/listen
DEEPGRAM_MODEL_DEFAULT=flux-general-en
DEEPGRAM_ENCODING_DEFAULT=linear16
DEEPGRAM_SAMPLE_RATE_DEFAULT=16000
DEEPGRAM_EOT_THRESHOLD_DEFAULT=0.7
DEEPGRAM_EOT_TIMEOUT_MS_DEFAULT=5000

# TTS-specific
DEEPGRAM_TTS_MODEL_DEFAULT=aura-2-thalia-en
DEEPGRAM_TTS_SAMPLE_RATE_DEFAULT=48000
```